Week 11 · Shipping it, and keeping it standing

Technical Interaction Design · ITU · Autumn 2026

Built 12 November 2026 · commit `af7bcaa`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-11.pdf

CONTENTS

Installing Your App on a Phone

- An installable app is a website that describes itself to the phone
- “Progressive” means each layer is optional
- The idea is Apple’s, the name is Google’s, and the standard belongs to all the browsers
- The manifest tells the phone the app’s name, its icon, where it starts, and how it opens
- DevTools tells you whether your app is installable, and why not
- On a phone, installing is hidden in the browser’s menu
- Without a browser around it, your app has to take over six of the browser’s jobs
- A service worker is what makes an app work offline, and you can skip it
- Notes
- Appendix: making todo-26 installable

When Users Break Your App

- Users find inputs you never thought of as inputs
- You find out only because the app reports its own errors
- A limit has to be chosen, enforced on the server, and explained on the client
- Some users cost you money, and some users are scripts

Running Code Server-Side

- Building turns your code into plain files, and deploying copies them somewhere public
- The static host runs none of your code, so it can keep none of your secrets
- The phone runs your code, and the user owns the phone
- A rule belongs on the server if it must hold no matter what the user does
- A backend runs your code next to the data, in triggers and in functions
- Essential complexity belongs to the problem; accidental complexity belongs to the tools
- The first job of server code is to say no
- A secret key stays on the server, and the app asks the server to use it
- Statistics over everyone’s data are computed where the data is
- An `afterSave` trigger cannot refuse a save; it carries out its consequences
- A pointer has no integrity, so a delete does the cleanup by hand
- Appendix: the Parse mechanics

Efficient Communication With the Backend

- Every request costs time, and on a hosted backend it also costs money
- One query, then one more for each result: the N+1 problem wastes the most requests
- Related objects can come back in the same request, as in a SQL join
- Fields the screen does not show are bandwidth wasted
- Shared data changes while the user looks at it
- Appendix: in Parse

Installing Your App on a Phone

Source: Lectures/Technical/Tooling/Installing-Your-App-on-a-Phone.md — something unclear? Open an issue, or send a pull request.

Since week 6 your app has a public address, and you can open it on your phone (see Publishing Your App on GitHub Pages). It works, but it is still a web page in a browser tab: an address bar on top, browser buttons at the bottom, and to find it again you have to find the tab.

Your projects (meal plans, grocery lists, households, record crates) are things people reach for on their phone, several times a day. This note makes the app installable: an icon on the home screen that opens full-screen, like an app from the store. An app that does this is called a **Progressive Web App** (PWA).

An installable app is a website that describes itself to the phone

No app store, no Swift or Kotlin, no second codebase. The phone needs three things before it offers to install a website:

1. the site is served over **HTTPS**, which GitHub Pages already does;
2. the page links to a **web app manifest**, a small file that describes the app;
3. the manifest lists **icons** large enough for a home screen.

That is enough for Chrome on Android and for Safari on iOS. The rest of this note is what those three things mean, and what changes for your app once it runs without a browser around it.

Something unclear in this section? Open an issue.

“Progressive” means each layer is optional

A PWA is not a different kind of app. It is your website, with layers added on top:

layer	what it adds	what it takes
website	runs in a browser tab	what you have
installable	home-screen icon, own window	a manifest and icons
works offline	starts without a network	a service worker
app-like features	push notifications, badges	more APIs, and permission from the user

A browser that does not support a layer still shows the website underneath, so nothing breaks. That is the *progressive* part: you go as far up as your app needs. For your projects, the second row is the one that matters.

Something unclear in this section? Open an issue.

The idea is Apple's, the name is Google's, and the standard belongs to all the browsers

When the iPhone launched in 2007, it had no App Store: web apps in Safari were *the* way to build for it, and Safari's "Add to Home Screen" and its `apple-touch-icon` date from then. A year later the App Store opened, and native apps took over.

The idea came back in 2015, when Alex Russell, an engineer on Chrome, and the designer Frances Berriman named it *Progressive Web Apps*, and Google pushed it in Chrome and Android. The pieces it rests on, the manifest and the service worker, are W3C standards, written together by the browser makers. Apple followed slowly: service workers reached Safari in 2018, push notifications for installed web apps only in 2023.

Something unclear in this section? Open an issue.

The manifest tells the phone the app's name, its icon, where it starts, and how it opens

The manifest is a JSON file, served next to your page. todo-26's:

```
{
  "name": "ToDo26",
  "short_name": "ToDo26",
  "start_url": ".",
  "scope": ".",
  "display": "standalone",
  "background_color": "#f6f3ee",
  "theme_color": "#f6f3ee",
  "icons": [
    { "src": "icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icons/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

- `name` is shown when installing; `short_name` goes under the icon, where there is room for about twelve characters.
- `start_url` is the page the icon opens; `scope` is the part of the site that belongs to the app; a link that leads outside it opens in a browser again.
- `display: "standalone"` is what removes the browser around the app. It is the line that makes it feel like an app.
- `theme_color` tints what the system draws around your app: the status bar, the title bar of the app's window, the address bar in Chrome on Android. `background_color` fills the screen for the moment before your app has drawn anything. Take both from your design system, so the app looks like itself from the first frame.
- `icons`: two square PNGs, 192 and 512 pixels. Keep the important part away from the edges, because Android cuts icons into circles or rounded squares.

The file's official name is the *Web Application Manifest*, not "PWA manifest". It describes a web app to the operating system, and it predates the term PWA; installing is simply what it is used for most.

Something unclear in this section? Open an issue.

DevTools tells you whether your app is installable, and why not

On your laptop, open the published app in Chrome, then **DevTools → Application → Manifest**. Chrome shows the manifest as it understood it, the icons it found, and under **Installability** anything that stops the app from being installed, with the reason. A wrong path shows up here long before it shows up as “nothing happens” on a phone.

This is the step to insist on even when an agent writes the manifest for you: the agent cannot see whether the phone accepted it.

Something unclear in this section? Open an issue.

On a phone, installing is hidden in the browser’s menu

- **Android, Chrome:** the ⋮ menu → **Add to home screen**, then **Install**.
- **iPhone, Safari** (and other browsers since iOS 16.4): the Share button → **Add to Home Screen**.

The icon lands on the home screen, and tapping it opens the app with no address bar. Your data is all there: the installed app talks to the same backend, in the same way, as the browser tab did.

Something unclear in this section? Open an issue.

Without a browser around it, your app has to take over six of the browser’s jobs

In a tab, the browser does a lot for your app that you never had to think about. Standalone mode takes the browser away, and with it these jobs:

the browser’s job	gone in the installed app	what your app has to do
back	no back button, and on iOS often no way back at all	a way back on every screen
reload	no reload button	pull to refresh, or reload on return
the address	no address bar to copy a link from	a share button where sharing matters
showing progress	no loading bar while a page loads	its own loading state
showing failure	no “can’t connect” page	its own failed state, with a way to retry
leaving	links to other sites take the user out of your app	treat outside links as a deliberate exit

Install your group’s app and use it for a day. Every time you reach for a browser button that is not there, write it down: that is your list of things to build.

Every screen needs a way back, because the phone may not offer one

On iOS, an installed web app has no back button. A list page with no link back to the home page is a dead end: the only way out is to close the app. todo-26's `← All lists` link was a nicety in the browser; in the installed app it is the only exit. Android has a system back button, but users of both phones should find the same way out in your interface.

Data other people change needs a way to reload: pull to refresh, or reload on return

If someone else adds an item to a shared list, the installed app does not show it until it loads the list again, and there is no reload button to press.

The usual answer is a gesture users already know from mail and social apps: pull the top of a list down, see a spinner, let go, and the list reloads. It belongs at the top of lists that show data other people change.

An even cheaper answer needs no gesture at all: reload when the user comes back to the app. People switch away from an installed app and back all the time, and the moment they return is exactly when the data is most likely to be stale. For a shared grocery list, that alone fixes most of the problem.

A link worth sharing needs a share button, because there is no address bar to copy it from

In a tab, sharing a list means copying the URL. Installed, the URL is invisible. If your app's sharing works through links (an invitation, a recipe, a list), it needs a share button. Phones offer a system share sheet that web apps can open, so the button can hand the link to messages, mail or chat like any native app does.

Loading and failure are yours to show, because the browser no longer shows them

The browser's loading bar and its "can't connect" page belonged to the tab. In the installed app, a slow request is a frozen screen and a failed one is a blank screen, unless your app shows otherwise. This is The Three States of Remote Data again, and now no browser covers for a missing state.

A link to another site is an exit from your app

A link that leads outside the app's scope opens in a browser, on top of or instead of your app. That is fine for "open the recipe's original page", and surprising for anything the user expected to stay inside. Decide for each outside link whether leaving is what the user wants.

Something unclear in this section? Open an issue.

A service worker is what makes an app work offline, and you can skip it

Open the installed app in airplane mode, and you get the browser's "you are offline" page: the app's files come from the static host every time, and the host is out of reach.

A **service worker** is a script the browser runs in the background, between your app and the network. It can keep copies of the app's files, so that the app at least starts without a connection.

Even then, the app would start and show **no data**: your data lives on the backend, and the backend is on the network too. The app would be in the `failed` state from The Three States of Remote Data, and it had better say so. Real offline support (saving changes locally and sending them later) is a large piece of work, and not what your projects need. An installable app that needs a connection is already a big step up from a browser tab.

Something unclear in this section? Open an issue.

Notes

- **The favicon is not the app icon.** The small icon in the browser tab comes from `<link rel="icon">` in `index.html`. Vite's starter template puts its own logo there, which is why so many student apps show a purple lightning bolt. Replace it with your app's icon.

Something unclear in this section? Open an issue.

Exam Questions

1. What does a website need before a phone will offer to install it? Which of those does GitHub Pages already give you?

2. What does “progressive” mean in Progressive Web App?

3. Why does installing a web app work differently on an iPhone than on an Android phone?

4. Which of the browser’s jobs does your app have to take over once it is installed? For three of them, say what your app has to do.

5. A service worker can make your app start without a network. Why would todo-26 still be of little use offline?

Appendix: making todo-26 installable

> In **todo-26**: pull request #2, live at https://itu-tid.github.io/todo-26/.

Every path in the manifest is relative. todo-26 is served at `https://itu-tid.github.io/todo-26/`, not at the root of the domain. A manifest saying `"start_url": "/"` would open the installed app on GitHub’s root page. Paths in a manifest are resolved relative to the manifest itself, so `"."` means “the folder I am in”, and the same file works on `localhost`, on GitHub Pages, and on a domain of your own. It is the same reason as `base` in week 6.

The pull request changes three things:

1. **`public/manifest.webmanifest`**, the manifest above. Vite copies everything in `public/` into the build unchanged, which is what a manifest and its icons need.
2. **`public/icons/icon-192.png`** and **`icon-512.png`**, exported as square PNGs.
3. **Three lines in the `<head>` of `index.html`:**

   ```
   <link rel="manifest" href="/manifest.webmanifest" />
   <link rel="apple-touch-icon" href="/icons/icon-192.png" />
   <meta name="theme-color" content="#f6f3ee" />
   ```

   These paths start with `/`, because Vite rewrites the links in `index.html` with `base` when it builds: `dist/index.html` says `/todo-26/manifest.webmanifest`. The `apple-touch-icon` line is optional: Safari has read the manifest’s icons since iOS 15.4, and the line only matters for older iPhones or a different icon on iOS.

For a service worker, the usual route with Vite is the `vite-plugin-pwa` package (read Libraries before adding it).

Something unclear in this section? Open an issue.

References

- Making PWAs installable, MDN
- Web application manifest, MDN

When Users Break Your App

Source: Lectures/Technical/Tooling/When-Users-Break-Your-App.md — something unclear? Open an issue, or send a pull request.

Stories from Zeeguu, a language-learning app, about inputs you did not think of as inputs, about how you find out that something broke, and about users who cost you money. This note is the problem; Running Code Server-Side is the fix.

Users find inputs you never thought of as inputs

Two stories from Zeeguu. The first is the one you expect; the second is the one worth learning from.

Some users type extreme values on purpose

Zeeguu lets learners choose how many words they want to be learning at the same time. One user typed a 1 followed by fifty zeros into that box.

This is the familiar case. A field accepts a number, so someone tries the largest number they can type, out of curiosity rather than malice. The fix is the one you already know: validate the input, and decide what the largest sensible value is before a user decides it for you.

Other users produce extreme values just by using the app a lot

The second story is more useful, because the user did nothing unusual.

When you click an article in the Zeeguu feed, the app also records which articles you scrolled past to get there, so it can learn which sources you keep ignoring. It sends that list to the server as a JSON array of ids, which the server stores in a database column that holds 4096 characters.

The feed loads 20 articles at a time. One user scrolled through roughly 80 pages, about 1,600 articles, and then clicked one. The list was about 12,700 characters long. The database refused it, the request failed, and the click was lost. The user clicked twice more and those were lost too.

Nobody typed anything extreme. The extreme value was built by *our* code, from ordinary behavior repeated more times than we had imagined. No form validation would have caught it, because there was no form.

Something unclear in this section? Open an issue.

You find out only because the app reports its own errors

Nobody told us about the second story. We found out because Zeeguu sends its own errors to Sentry, an error-monitoring service.

Your users will not tell you when things don't work

The user saw nothing: the click just did not register. If the app had not been reporting its own errors, the bug would still be there.

Error monitoring tools such as Sentry fix this. You add a few lines to your app:

```
import sentry_sdk
sentry_sdk.init(dsn="https:// ...")
```

and from then on every unhandled exception is sent to a dashboard, along with the stack trace, the request, the browser and operating system, and the values of local variables. Sentry also works in the browser, for JavaScript errors that never reach your server at all.

For the story above, the Sentry report said, in one screen: this error, three times, on 5 October, from one person on Safari on a Mac, at this line of our code, with this SQL statement and these parameters. The 12,700-character list was right there in the parameters, which was the whole diagnosis. We did not have to reproduce the bug to understand it.

Error reports, session recordings and your own logs answer different questions

Error monitoring answers *what broke*. Two other tools answer questions it cannot:

- **Session recording** (tools such as Contentsquare, formerly Hotjar, or Microsoft Clarity) records what users did on the page: where they clicked, how far they scrolled, where they hesitated. It answers *where do people get stuck*, which no error report shows, because getting stuck is not an error. It also records whatever is on the screen, so the same care about personal data applies, twice over.
- **Your own logs**: a table where your app writes the events you care about, such as "list shared", "translation requested", "item bought". It answers *what are people actually doing with the app*, in your own terms, and it is the only one of the three you fully control. The per-day translation limit below is such a log: to count a user's translations, you have to have written them down.

For a user study, the third is the most useful: you can count how often the feature you built is used, instead of asking.

Something unclear in this section? Open an issue.

A limit has to be chosen, enforced on the server, and explained on the client

The fix for the second story, and for every value that grows with use.

Anything that grows with use needs a limit you chose

List lengths, history, retries, upload sizes, session lengths: anything that grows each time the user does something will one day be bigger than you expected. “Nobody will scroll that far” is an assumption, and the code relies on it whether you wrote it down or not.

If you do not choose the limit, something else chooses it for you. Here it was a database column width, and it was enforced by crashing.

Choose the limit from what the data is for

The scrolled-past list exists to detect sources a user skips. An article 1,500 cards above the click, flicked past in a fast scroll, says nothing about what the user wants. Keeping only the last 100 or so ids before the click keeps all of the meaning and fits the column with room to spare.

So the right limit came from asking what the data means, not from looking up how much the column can hold.

Only a limit on the server holds

Capping the list in the web app would help. But the iOS and Android apps already installed on people’s phones keep sending the old, uncapped lists until each user updates, which can take months. The client can be polite about limits; the server is the only place where a limit actually holds.

The frontend explains a limit; the backend enforces it

Every limit therefore lives in two places, and each has a different job.

The backend enforces. It refuses whatever breaks the rule, from any client, in any version, including a script that never saw your form. It does not need to be friendly; it needs to be there.

The frontend explains. It is where the user meets the limit, so it is where the limit has to make sense:

- **before** they hit it: a field that says *0–100 words*, a counter that says *180/200*, a list that says *12 of 500 items*;
- **while** they approach it: a warning as the counter nears the end, not a surprise after pressing Save;
- **when** they hit it anyway: a message in the user’s terms (*“A list can hold at most 500 items”*), not the server’s (*“Error 142”*), and without throwing away what they typed.

A limit that only the backend knows feels like a bug to the user: the button worked yesterday, and today it silently does nothing. A limit that only the frontend knows is not a limit at all. So write it twice, with the same number, and when it changes, change both.

Form validation is the frontend's half of a limit

Since week 2, todo-26's Add button is disabled while the field is empty: `disabled={text.length === 0}`. That is form validation, the smallest kind. The same form can explain the rest of its limits.

The browser checks simple rules by itself, and says so in its own words

HTML inputs take their rules as attributes, and the browser enforces them before the form is submitted:

```
<input required maxLength={200} />          // not empty, at most 200 characters
<input type="email" required />             // looks like an email address
<input type="number" min={0} max={100} />   // a number within a range
```

It is free, and it works without any JavaScript. But the browser chooses the wording and the moment, and some rules are silent: `maxLength` just stops accepting keys, so the user does not learn why their typing stopped.

A controlled input can explain a limit before the user reaches it

Because the text is state (Forms and Controlled Components), anything about it can be shown while the user types, and none of it needs new state:

```
const MAX = 200;

<input value={text} onChange={(e) => setText(e.target.value)} maxLength={MAX} />
<small>{text.length}/{MAX}</small>
<button disabled={text.trim().length === 0}>Add</button>
```

The counter is derived from `text`, like the disabled button. It turns a silent `maxLength` into a limit the user can see coming.

An error shown on the first keystroke is stupid and anti-usability!

"Not a valid email" appearing after the user typed one letter is not help: they were not finished. Show a field's error when the user leaves the field or presses submit, and from then on update it as they type, so that it disappears the moment the value becomes valid.

A failed save must not throw away what the user typed

todo-26's `AddForm` clears the field right after handing the text to `onAdd`, before anyone knows whether the save worked:

```
onAdd(task);
setTask("");   // gone, even if the server refuses it a second later
```

When the server refuses (the list is full, the text too long), the user's input is already gone. Clear the field only after the save succeeded, and show the server's reason when it did not:

```
try {
  await onAdd(task);
  setTask("");
} catch (error) {
  setError(error.message); // e.g. "A list can hold at most 500 to-dos"
}
```

The server's message is the backend's half of the limit reaching the user; Running Code Server-Side shows where it comes from.

Something unclear in this section? Open an issue.

Some users cost you money, and some users are scripts

So far the extreme values were accidents. The next ones cost you money, or come from someone who never opened your form.

Anything that costs you money per use needs a limit per user

Some of what your app does is free to you. Some is not. Zeeguu translates any word a learner taps, and every translation is a request to a paid translation service. One user tapping a thousand words a day is a learner; a script tapping a million is a bill. So Zeeguu limits **how many translations a user can ask for per day**. Nobody reading normally comes near the limit, and a runaway client or a misused account stops at it. The next note shows what such a limit looks like in code: about ten lines that count today's translations before calling the service.

The same holds for anything metered: AI calls, text messages, emails, storage, even your free tier's request count. For each one, decide what a heavy but honest user needs, and stop there.

Creating an account is free, so somebody will create a million

Zeeguu also limits **how many accounts can be created from the same network address** in a given time. Without such a limit, a script can create accounts as fast as the server answers: to spam other users, to get around the per-user limits above (a fresh account has a fresh daily quota), or just to fill your database.

The common defenses, from cheapest to most annoying for honest users:

- a limit on sign-ups per IP address per hour;
- a confirmation email that has to be clicked before the account works;
- a CAPTCHA on the sign-up form.

Your form adds one item at a time; a script adds a million

Look at your own app with these two in mind:

1. **How do you stop one user from creating a million to-dos (or dishes, or records) in their list?** Your form adds one at a time, but a script calls `save()` in a loop.
2. **How do you stop one person from creating a million accounts?** Your sign-up page is public, and so is the class that stores your users.

Neither has an answer on the client, for the reason in Running Code Server-Side: the user controls the client. Both answers are a few lines of server code, in the next note.

Something unclear in this section? Open an issue.

Exam Questions

1. A form field is validated to accept at most 1,000. Give an example of a value that still gets too large without any user typing it, and say where its limit should be enforced.

2. A bug causes a click to be silently ignored. The user does not complain. How would you find out it happened, and what would you need in the error report to fix it without reproducing it?

3. Error reports, session recordings and your own logs: which question does each answer?

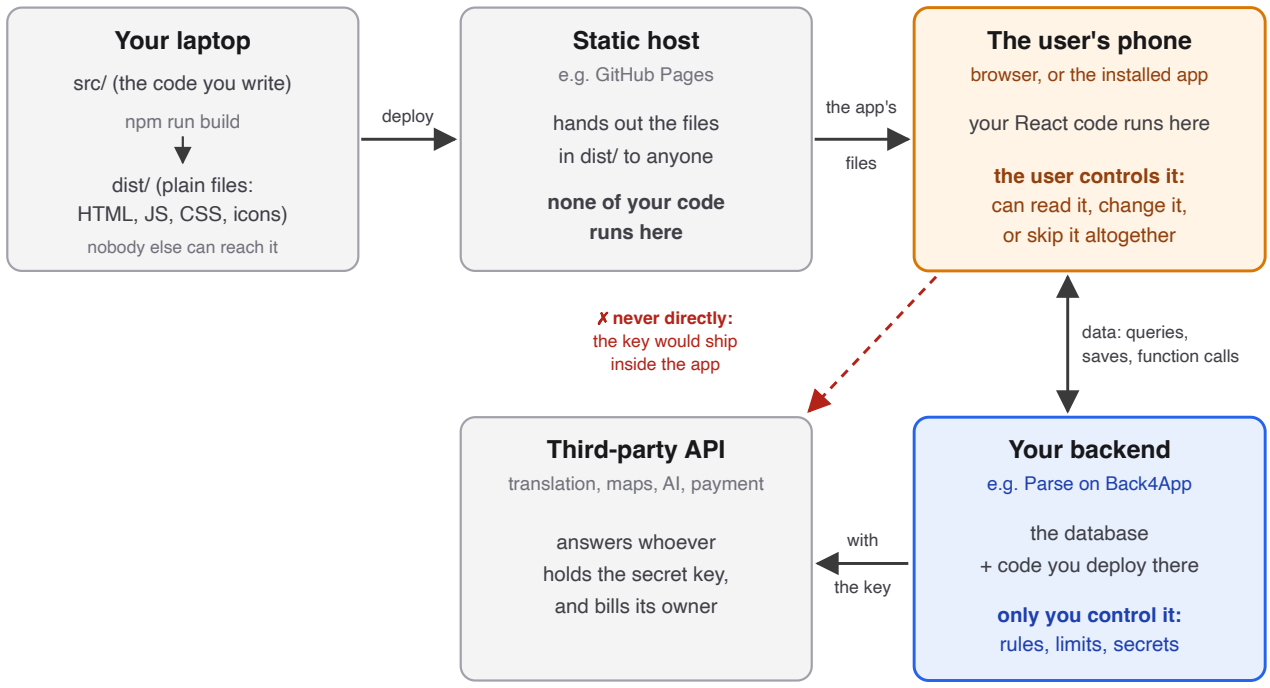
4. Why is it not enough to cap the scrolled-past list in the web app, when the web app is the one that builds it?

5. How would you stop one user from creating a million to-dos, and one person from creating a million accounts? Why is neither possible on the client alone?

Running Code Server-Side

Source: Lectures/Technical/Backend/Running-Code-Server-Side.md — something unclear? Open an issue, or send a pull request.

When Users Break Your App ended with limits that only hold on the server. Before writing any, one picture of where each part of your app runs, because which code goes where follows from it.



The orange box belongs to the user; the blue box belongs to you. A rule that must hold goes in the blue box.

Building turns your code into plain files, and deploying copies them somewhere public

What you write is not what you ship. The build takes your components, your imports and your libraries, and produces a folder of plain files that any browser understands: one HTML page, a few bundled JavaScript and CSS files, and whatever images and icons you put next to them. Deploying copies that folder to a computer that is always on and that anyone can reach.

You have done this since week 6 (Publishing Your App on GitHub Pages). The tools are a detail: a different bundler or a different host changes the commands, not the picture.

Something unclear in this section? Open an issue.

The static host runs none of your code, so it can keep none of your secrets

GitHub Pages, and every host like it, does one thing: when a browser asks for a file, it sends the file. It does not run your JavaScript; it does not know what your app does. That is why it can be free, and why anything that can serve files can host your app.

It also means there is nowhere on it to keep a secret. Everything in the folder you deployed is readable by anyone who asks for it.

Something unclear in this section? Open an issue.

The phone runs your code, and the user owns the phone

Your React code runs on the user's device. That is what makes the app fast and pleasant: a click changes the screen without asking anyone.

It also means the code is the user's to do with as they like. They can open the developer tools and read it, including the Parse keys in it. They can change a variable and click again. They can skip your app entirely and send requests to your backend from a script, with values your form would never have let through. A check written in a component is a check *the user agreed to run*.

Something unclear in this section? Open an issue.

A rule belongs on the server if it must hold no matter what the user does

That gives the principle for deciding where a piece of code goes:

> **The client belongs to the user; the server belongs to you. Anything that must hold no matter what the user does runs on the server. Anything that only makes the experience better can run on the client.**

On the server, then:

- **rules and permissions**: who may read or change what (the ACLs you set since week 5 are such rules: the server enforces them);
- **limits**: how long a name, how many items, how many requests per day;
- **secrets**: any key that costs money or grants access, such as the key to a translation or AI service;
- **other users' data**: statistics over everyone, which no single user may download;
- **things that must happen even if the app is closed**: a log entry when a task is done, a cleanup when a list is deleted.

On the client: instant feedback, a disabled button, a character counter, a friendly message. These should **repeat** what the server enforces, so the user hears "too long" before sending rather than

after; they never replace it.

When Users Break Your App showed what happens when they are not. The rest of this note shows how each is written.

Something unclear in this section? Open an issue.

A backend runs your code next to the data, in triggers and in functions

A backend like Parse lets you deploy small pieces of JavaScript that run **on the server**, where the user cannot reach them. They come in two kinds:

- **triggers**, which run by themselves before or after something is saved or deleted;
- **cloud functions**, which your app calls by name, like a function that happens to run on another computer.

Something unclear in this section? Open an issue.

Essential complexity belongs to the problem; accidental complexity belongs to the tools

In *No Silver Bullet* (1986), Fred Brooks split the complexity of software in two. **Essential** complexity is in the problem itself: a list holds at most 500 items; a translation costs money. **Accidental** complexity comes from the tools we solve it with: what the trigger is called, where the object is found in the request, which option skips the permission check.

Every example below is a few lines of essential rule wrapped in Parse's accidental ones. The rule would survive a move to Firebase or to your own server; the wrapping would not. When you read one, find the rule first. The Parse mechanics are collected in the appendix.

Something unclear in this section? Open an issue.

The first job of server code is to say no

The most common server code is a check before a write. In Parse it is a `beforeSave` trigger: it runs before every save of an object of that class, from any client, and if it throws, the save does not happen.

A check that only the client makes is a suggestion

```
Parse.Cloud.beforeSave("TodoItem", (request) => {
  const text = request.object.get("text");

  if (!text || text.trim().length === 0) {
    throw new Parse.Error(Parse.Error.VALIDATION_ERROR, "A to-do needs a text");
  }
  if (text.length > 200) {
    throw new Parse.Error(Parse.Error.VALIDATION_ERROR, "A to-do can be at most 200
        characters");
  }
});
```

Your form probably already refuses an empty to-do. Keep that: it tells the user before they press the button. But the form only runs if the user uses your form. The trigger runs for every save, whoever sends it.

A per-user limit counts before it lets a save through

The first question from the previous note: how do you stop one user from putting a million to-dos in a list? Count what is already there:

```
Parse.Cloud.beforeSave("TodoItem", async (request) => {
  if (request.original) return; // an edit, not a new to-do: nothing to count

  const count = await new Parse.Query("TodoItem")
    .equalTo("list", request.object.get("list"))
    .count({ useMasterKey: true });

  if (count >= 500) {
    throw new Parse.Error(Parse.Error.VALIDATION_ERROR, "A list can hold at most 500 to-dos");
  }
});
```

Choose the number the way When Users Break Your App chose it: from what the data is for. A shopping list with 500 items is no longer a shopping list.

A limit on sign-ups counts by network address

The second question: a million accounts. A new user is a save too, of the special class `_User`, and the request knows the IP address it came from:

```
Parse.Cloud.beforeSave(Parse.User, async (request) => {
  if (request.original) return; // only sign-ups, not profile changes

  request.object.set("signUpIp", request.ip);

  const oneHourAgo = new Date(Date.now() - 60 * 60 * 1000);
  const recent = await new Parse.Query(Parse.User)
    .equalTo("signUpIp", request.ip)
    .greaterThan("createdAt", oneHourAgo)
    .count({ useMasterKey: true });

  if (recent >= 5) {
    throw new Parse.Error(Parse.Error.VALIDATION_ERROR, "Too many new accounts from this
        network. Try again later.");
  }
});
```

Five an hour is plenty for a family signing up together, and nothing for a script. A whole class on the ITU network shares one address, though: think about who your limit hits besides the attacker.

A toast tells the user what the server refused, without stopping them

A refused save reaches your app as an error from `save()`. What the user sees is up to you, and `alert()` is the worst choice: it stops everything, looks like the browser's and not your app's, and must be clicked away.

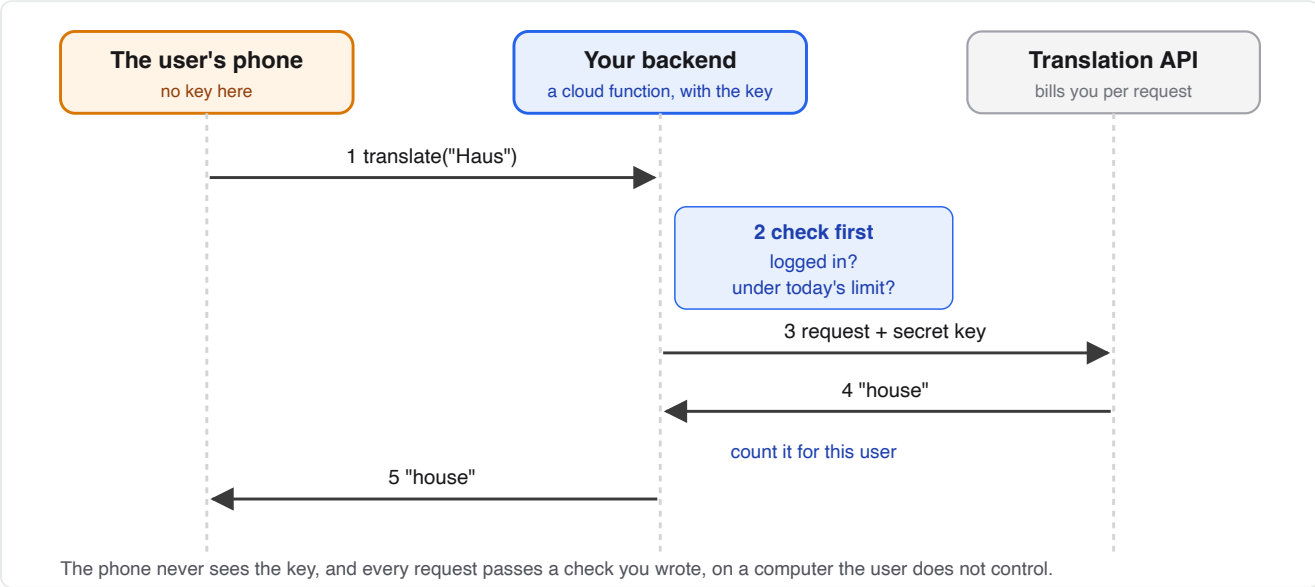
The usual pattern is a **toast**: a short message that slides in at the edge of the screen, says what happened ("A list can hold at most 500 to-dos"), and disappears by itself after a few seconds, without blocking anything. Use it for messages the user should notice but need not act on. If they must decide something, use a dialog; if the problem belongs to one field, put the message next to the field. In each case, keep what the user typed, so a refused save does not also cost them their input.

Something unclear in this section? Open an issue.

A secret key stays on the server, and the app asks the server to use it

A translation service, a maps API, an AI model: each gives you a **secret key**, and bills you for every request made with it. Put the key in your app and it ships to every phone (the static host cannot keep secrets); anyone can copy it and spend your money.

So the app does not call the service. It calls a cloud function on your server, which checks the request, calls the service with the key, and passes the answer back:



From the app, a cloud function is called by name:

```
const translation = await Parse.Cloud.run("translate", { word: "Haus" });
```

On the server, step 2 of the diagram is the part you are paid for; steps 3 and 4 are plumbing:

```
Parse.Cloud.define("translate", async (request) => {
  if (!request.user) {
    throw new Parse.Error(Parse.Error.INVALID_SESSION_TOKEN, "Log in to translate");
  }

  // 2: under today's limit? Every translation was written down, so we can count them.
  const startOfToday = new Date();
  startOfToday.setHours(0, 0, 0, 0);
  const usedToday = await new Parse.Query("Translation")
    .equalTo("user", request.user)
    .greaterThan("createdAt", startOfToday)
    .count({ useMasterKey: true });
  if (usedToday >= 100) {
    throw new Parse.Error(Parse.Error.VALIDATION_ERROR, "That's 100 translations today. More
        tomorrow!");
  }

  // 3 and 4: the service, with the key that only the server knows
  const response = await fetch("https://translation.example.com/v1/translate", {
    method: "POST",
    headers: { Authorization: `Bearer ${process.env.TRANSLATION_API_KEY}` },
    body: JSON.stringify({ text: request.params.word, to: "en" }),
  });
  const { translation } = await response.json();

  // count it for this user
  const log = new Parse.Object("Translation");
  log.set("user", request.user);
  log.set("word", request.params.word);
  await log.save(null, { useMasterKey: true });

  return translation;
});
```

`translation.example.com` stands for whichever service you use; its documentation says what to send.

Something unclear in this section? Open an issue.

Statistics over everyone's data are computed where the data is

Some answers need every user's data: the average time spent on a task, how many lists were created this week. The app cannot compute them, because a user may not download everyone else's to-dos, and should not have to. A cloud function can: it runs with full access, computes the number, and returns only the number.

```
Parse.Cloud.define("weeklyStats", async () => {
  const oneWeekAgo = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);
  const query = new Parse.Query("TodoItem").greaterThan("createdAt", oneWeekAgo);

  const created = await query.count({ useMasterKey: true });
  const done = await query.equalTo("done", true).count({ useMasterKey: true });

  return { created, done };
});
```

If such a number is expensive and asked for often, compute it once a day and store it, instead of on every request.

Something unclear in this section? Open an issue.

An `afterSave` trigger cannot refuse a save; it carries out its consequences

An `afterSave` trigger runs after a save has succeeded. It cannot refuse anything; it is for consequences. Here, every time a to-do is ticked off, the server writes a log entry, whichever client ticked it:

```
Parse.Cloud.afterSave("TodoItem", async (request) => {
  const justDone =
    request.object.get("done") === true &&
    request.original?.get("done") === false;
  if (!justDone) return;

  const entry = new Parse.Object("CompletionLog");
  entry.set("list", request.object.get("list"));
  entry.set("doneBy", request.user);
  await entry.save(null, { useMasterKey: true });
});
```

Exercise. Mark the lines of this trigger that state the rule (the essential complexity), and the lines that are there because it is Parse (the accidental complexity). Then write the rule in one sentence, without any code. Is the sentence complete? (Hint: what does "just" in *just ticked off* need to be computed from?)

Something unclear in this section? Open an issue.

A pointer has no integrity, so a delete does the cleanup by hand

A Parse pointer is not a checked foreign key (see Relationships Between Object Classes): delete a list, and its to-dos stay behind, pointing at nothing. Parse has no cascading deletes either. A `beforeDelete` trigger is where you put that rule back:

```
Parse.Cloud.beforeDelete("List", async (request) => {
  const todos = await new Parse.Query("TodoItem")
    .equalTo("list", request.object)
    .find({ useMasterKey: true });
  await Parse.Object.destroyAll(todos, { useMasterKey: true });
});
```

The other policy is to refuse: `throw` if the list still has to-dos, and the delete does not happen. Either way the rule now lives on the server, where no client can skip it.

A relational database cascades by itself, and code still has work to do before a delete

In the relational databases from your database course, this rule is not code: it is part of the schema. A foreign key declared `ON DELETE CASCADE` removes the to-dos with their list; `ON DELETE RESTRICT` refuses the delete. Parse makes you write by hand what SQL gives you in one clause.

But even with a relational database, some things have to happen before a delete, and the schema cannot do them:

- **things outside the database**: a list's uploaded photos live in file storage, which no foreign key reaches;
- **undo**: many apps never really delete; they mark the row as deleted (a *soft delete*) and hide it, so that "Undo" and "Restore from trash" are possible;
- **history**: an archive or an audit log of what was deleted, by whom, and when;
- **rules no key can express**: you cannot delete the last admin of a household; a list shared with others warns them first.

Something unclear in this section? Open an issue.

Exam Questions

1. When should code run on the server rather than on the client? State the principle, and give three kinds of code that follow from it.

2. Why can a static host like GitHub Pages be free, and why can it never keep a secret for your app?

3. Your form refuses empty to-dos. Why do you still need this trigger?

```
Parse.Cloud.beforeSave("TodoItem", (request) => {
  const text = request.object.get("text");
  if (!text || text.trim().length === 0) {
    throw new Parse.Error(Parse.Error.VALIDATION_ERROR, "A to-do needs a text");
  }
});
```

4. What is the difference between `beforeSave` and `afterSave`? Give one use of each.

5. How do you stop one user from creating a million to-dos? How do you stop one person from creating a million accounts?

6. Your app uses a paid translation service. Why can the app not call it directly, and what does the cloud function in between do besides hiding the key?

7. What are essential and accidental complexity? Point to one of each in the `beforeDelete` trigger above.

8. In a relational database, deleting a list can delete its to-dos without any code. Name two things that still need code before a delete.

Appendix: the Parse mechanics

Where the code goes. On Back4App: **Dashboard → Cloud Code**, edit `cloud/main.js`, and press **Deploy** (see Back4App's guide). The code then lives in the dashboard, not in your repository, so nobody reviews it in a pull request. Keep a copy of `cloud/main.js` in your repository and paste from there, or deploy from the command line with Back4App's CLI.

One trigger per class and event. Parse keeps one `beforeSave` per class: registering a second replaces the first. In your project, the validation and the limit for `TodoItem` go in the same function.

What the request carries:

`request.object`	the object being saved or deleted, with the new values
`request.original`	the object as it was before this save; `undefined` when it is new
`request.user`	the logged-in user who sent the request, if any
`request.params`	the parameters a cloud function was called with
`request.ip`	the address the request came from

`useMasterKey: true` lets server code read and write past ACLs and class-level permissions. The counts above need it, because they must see objects the requesting user may not. It works only on the server; never put the master key in your app.

Secrets such as `TRANSLATION_API_KEY` go in the server's environment variables (on Back4App, in the app's server settings), and are read with `process.env`. They never appear in `main.js`, which you may want to keep in a public repository.

Built-in rate limits. Parse Server also has a `rateLimit` option that caps requests per IP address for a function or a class. It is configured on the server rather than written as a trigger; the hand-written count above shows what such an option does.

Protect what you store about users. `signUpIp` is personal data on a class that sign-up makes partly readable (Sharing a List). Add it to the protected fields, so only the server sees it.

Something unclear in this section? Open an issue.

Efficient Communication With the Backend

Source: Lectures/Technical/Backend/Efficient-Communication-with-the-Backend.md — something unclear? Open an issue, or send a pull request.

Your app talks to its backend over the network, and every conversation costs something: time for the user, and requests, bandwidth and money for you. This note is about having fewer, smaller conversations, and about how an app hears that something changed. None of it is specific to Parse; the appendix shows the Parse spelling.

Every request costs time, and on a hosted backend it also costs money

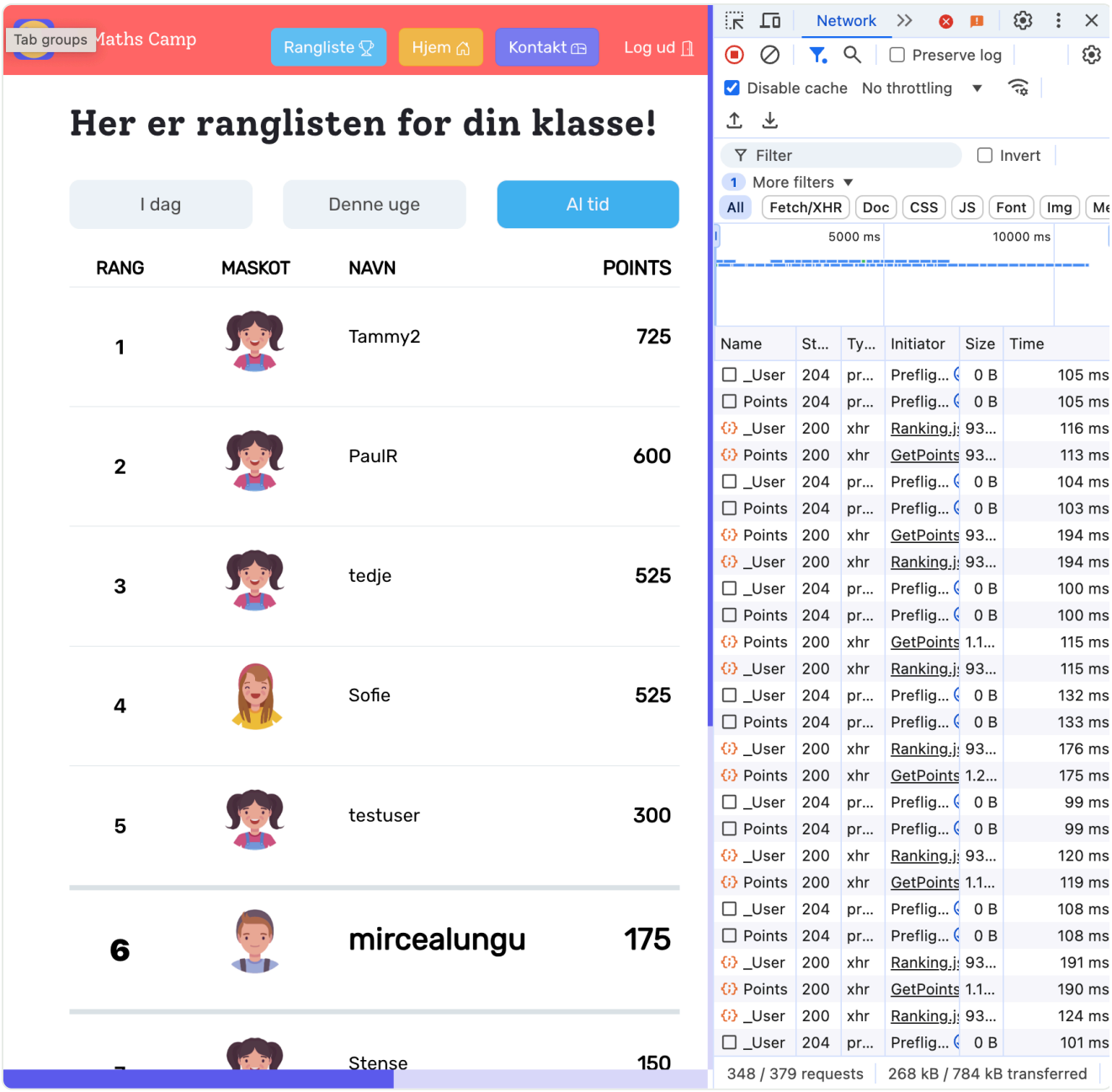
A backend-as-a-service like Back4App is a computer someone else runs for you, and they charge for it. The free tier you use limits:

- the number of requests,
- the data transferred,
- the data stored.

Full Features List	Free Start for Free	MVP $15/month per app Buy Now	Pay as you go $80/month per app Buy Now
Requests/Month	25k	500k	5M
Data Storage	250MB	2GB	4GB
Data Transfer	1GB	250GB	1TB
File Storage	1GB	50GB	250GB
Apps per subscription	1	1	1
Daily backup	✗	✓	✓

So the way your app talks to the backend is a design decision with two prices. Too many requests make the app slower *and* use up the quota faster. Usually fixing one fixes the other.

Here is the Network tab of a student app, MathsCamp, loading one screen:



Discussion: Why might we want to improve this? To save money, or to have a faster app? Which of the two would a user notice?

Something unclear in this section? Open an issue.

One query, then one more for each result: the N+1 problem wastes the most requests

The most common way to waste requests: you ask for a list of N things, and then, for each of them, you ask for one more thing. One request becomes N+1. With ten lists it is eleven requests; with a hundred, it is a hundred and one, each one a round trip the user waits for.

You have written one already, and you have already fixed one. In week 5, Loading the page, the simple way queried the lists, and then ran one more query *inside each list's block* for its to-dos. In week 6, the sharing queries asked for the memberships *together with* their lists and the lists' owners, so that "shared by ada" did not cost one extra request per list. That was the fix; this section gives the problem its name.

Something unclear in this section? Open an issue.

Related objects can come back in the same request, as in a SQL join

The fix is always the same: tell the backend, in the first request, what else you are going to need. A query for to-dos that will be shown with their list's name asks for the to-dos *including* their lists. The backend follows the references on its side, where it is fast, and sends everything back in one answer.

In SQL this is a `JOIN`. Every backend has a way to say it; in Parse it is `include` (see the appendix).

Something unclear in this section? Open an issue.

Fields the screen does not show are bandwidth wasted

The opposite waste: one request, but far more data than the screen needs. A list of to-do titles does not need each to-do's long description, its attached photos, or its history. Ask for the fields you will show, and the answer gets smaller and faster.

In SQL this is the difference between `SELECT *` and `SELECT title, done`. In Parse it is `select` and `exclude`.

Something unclear in this section? Open an issue.

Shared data changes while the user looks at it

For most screens, nobody else changes the data while it is shown: you are editing your own list. But when a screen shows data that *other people* change (the shared grocery list your flatmate is ticking off in the shop, the household's meal plan, a chat), the screen goes stale. There are two ways to keep it fresh.

Polling asks again every few seconds, and is usually enough

The simple way: set a timer, and when it fires, load the data again and show it. Nothing changes on the server; it is just the same query, repeated.

Every poll is a request, so poll only on screens that show shared data, only while they are open, and every few seconds rather than every few milliseconds. Cheaper still: reload when the user comes back to the app, which needs no timer at all.

In React, a timer started in an effect must be stopped when the component goes away, or every visit to the page starts one more timer that never stops:

```
useEffect(() => {
  const interval = setInterval(() => {
    // load the data again
  }, 10000);

  return () => clearInterval(interval); // the page is closed: stop asking
}, []);
```

Because that cleanup is so easy to forget, next week Custom Hooks writes it once, as `useInterval`.

Push lets the server announce a change, and is worth it only when seconds matter

The other way round: the app keeps a connection to the server open and **subscribes** to a query, and the server sends a message the moment something matching it is created, changed or deleted. Nothing is asked twice; the update arrives as soon as it happens.

It is the observer pattern across the network, and most backends offer it under their own name: Parse calls it *LiveQuery*, Firebase has `onSnapshot`, Supabase calls it *Realtime*. Underneath it is usually a **WebSocket**, a connection that stays open so that either side can send at any time, unlike an ordinary request, which the client always starts.

Push is the right tool when a few seconds of delay is visible: a chat, a multiplayer game, a cursor in a shared document. It is also more to set up, more to clean up, and more to debug. For a grocery list, nobody notices whether an item appears now or ten seconds from now. **Start with polling; move to push when a user can tell the difference.**

Something unclear in this section? Open an issue.

Exam Questions

1. What is the N+1 problem? Where did you write one, and how did you fix it?

2. What is wrong with this code, and which single line fixes it?

```
const query = new Parse.Query("TodoItem");
const todos = await query.find();

for (let todo of todos) {
  const list = todo.get("list");
  await list.fetch();
  console.log(list.get("name"));
}
```

3. What is the difference between polling and push? Why is polling usually enough for a shared grocery list, and when is it not?

4. Why ask for only some fields of an object, when one request brings the whole object anyway?

5. A page polls every ten seconds with `setInterval` in a `useEffect`, without a cleanup function. What happens after the user has opened and left that page five times?

Appendix: in Parse

Related objects: `include`. A pointer comes back as just an id. `include` asks Parse to send the object it points at too:

```
const query = new Parse.Query("TodoItem");
query.equalTo("done", false);
query.include("list");            // the list each to-do points at, in the same answer
const todos = await query.find();

todos[0].get("list").get("name"); // no second request
```

Pointers of pointers work with a dot: `include("list.owner")`.

Only some fields: `select` and `exclude`.

```
query.select("text", "done");     // only these fields
query.exclude("notes", "photo");  // everything except these
```

Use `select` when you need a few fields, `exclude` when you need all but a few large ones.

Push: LiveQuery. It has to be switched on for each class in the server settings, and then:

```
const subscription = await new Parse.Query("TodoItem")
  .equalTo("list", list)
  .subscribe();

subscription.on("create", (todo) => { /* add it to the screen */ });
subscription.on("update", (todo) => { /* replace it */ });
subscription.on("delete", (todo) => { /* remove it */ });

// and when the page closes:
subscription.unsubscribe();
```

See Live Queries in the Parse guide.

Something unclear in this section? Open an issue.